

A Developer's Guide to the ML Business Feasibility Predictor

How We Built a Trained Random Forest Pipeline to Predict Global Startup Success

Author: Portfolio Developer

Date: May 2026

Stack: Scikit-Learn | Streamlit | Plotly | Pandas | Joblib

Complexity: Intermediate Data Science

1. The Big Picture (What is this?)

Imagine you want to start a new business—say, a sourdough bakery in India or a software startup in Germany. Normally, entrepreneurs just guess if they have enough money and if they will succeed. This project is a **Machine Learning (ML) Decision Dashboard** that removes the guesswork.

It takes simple inputs from the user—like their business type, their starting capital, where they are launching, and their team size. It then passes those inputs through a **trained Machine Learning model** which instantly outputs two real-world metrics:

- **Survival Probability:** The likelihood (from 0% to 100%) that the business survives its first 3 years.
- **Expected Annual Net Income:** A realistic prediction of yearly profit or loss in local currency.

2. The Data Core & Economics

A machine learning model cannot make predictions without training data. Because real-world business failure data is highly private, we created a specialized economic engine (in **train_model.py**) that synthesizes **5,000 historical launch records** containing authentic, mathematical business relationships.

We also embedded a **Macroeconomic Database** built on real-world indices:

- **Purchasing Power Parity (PPP):** Measures the value of currencies. For example, \$1 in India buys much more than \$1 in the US. The model scales capital automatically using this factor.
- **Local Labor Index:** Scales staffing expenses depending on the country's average wage rates.
- **Commercial Rent Index:** Determines brick-and-mortar overhead costs dynamically.

3. How the Machine Learning Brain Works

We used a highly reliable and popular machine learning algorithm called a **Random Forest**. Instead of relying on a single 'decision tree' (which is prone to errors), a Random Forest constructs **120 independent decision trees** and averages their outputs for high accuracy.

We built two separate pipelines:

1. **The Classifier ('RandomForestClassifier'):** Predicts a binary target (1 for Success, 0 for Failure). It outputs the probability percentage that a startup survives.
2. **The Regressor ('RandomForestRegressor'):** Predicts a continuous value (Annual Profit or Loss amount).

Handling Categories Seamlessly (One-Hot Encoding):

Machine learning models only understand numbers, not text like 'SaaS' or 'Experienced'. We solved this by using Scikit-Learn's **OneHotEncoder**. This converts categorical words into binary columns (1s and 0s) under the hood using a unified **Pipeline** object, eliminating the risk of shape mismatch errors during predictions.

4. The Technical Code Architecture

If someone asks you how the project is organized, here is the file structure to show them:

File Name	Core Purpose / Function
requirements.txt	Specifies minimal libraries: streamlit, pandas, numpy, scikit-learn, joblib.
macro_data.csv	Locational indices mapping PPP, labor indices, and rent ratios.
train_model.py	Generates 5k startup rows, processes features, trains the Random Forest, and exports .pkl files.
app.py	The Streamlit dashboard. Collects inputs, runs the saved models, and prints custom diagnostics.
survival_model.pkl	The serialized binary of the trained Classifier model.
profit_model.pkl	The serialized binary of the trained Regressor model.

5. Step-by-Step Building Steps

Here is the order in which we built this application:

Step 1: Setting up the Data

We created a tabular CSV containing country multipliers. This allows the model to scale a startup in India differently than one in Canada automatically.

Step 2: Coding the Training Engine (`train\_model.py`)

We wrote a simulator that models realistic market failures (e.g. online stores failing if they have no marketing budget, cafes failing if they have low capital and high rent). We trained our models on this dataset and exported them.

Step 3: Creating the Interface (`app.py`)

We wrote the frontend in Streamlit, utilizing standard layout forms. We load the `.pkl` files and calculate expected profits on-the-fly.

6. How to Talk About This in an Interview

If a recruiter or peer asks about this project, explain it in three simple steps:

1. **The Problem:** '90% of startups fail due to poor cash flow management. I built an ML tool that evaluates startup ideas globally.'

2. **The Tech:** 'I built a Scikit-Learn Random Forest Pipeline trained on economic indices and 5,000 launch outcomes. By utilizing ColumnTransformers, the model handles categorical parameters natively without breaking.'
3. **The Outcome:** 'It is deployed on a sleek Streamlit dashboard, letting real users input their budgets and get dynamic survival percentages and localized profit forecasts.'